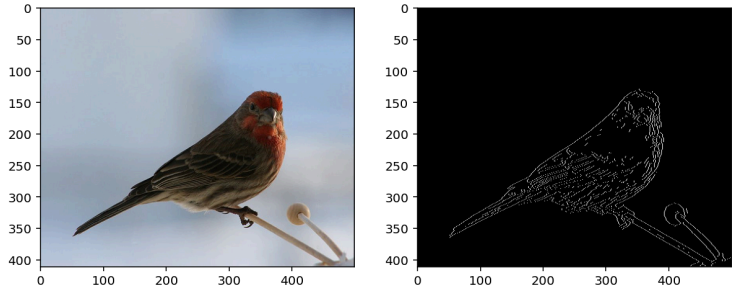
Classical_CV

1. Canny edge detection
    1. Code:

```
from PIL import Image
import numpy as np
import matplotlib.pyplot as plt
from filters import *
from utils import *

from typing import Tuple
from convolution import convolve


"""

Steps involved in Canny edge detection:
1. Noise reduction.
2. Gradient calculation.
3. Non-maximum suppression.
4. Double thresholding.
5. Edge tracking by Hysteresis.

References:
6. J. Canny, "A Computational Approach to Edge Detection," 1986
7. https://medium.com/towards-data-science/canny-edge-detection-step-by-step-in-python-computer-vision-b49c3a2d8123

"""


def filter_gaussian(
    image: NDArray[np.uint8], kernel_size: int, sigma: float
) -> NDArray[np.uint8]:
    C, H, W = image.shape
    gaussian_kernel = get_gaussian_kernel(
        kernel_size=kernel_size, num_channels=C, sigma=sigma
    )
    convolved_image = convolve(image=image, kernel=gaussian_kernel, stride=1)
    return convolved_image


def calculate_gradient(
    image: NDArray[np.uint8],
) -> Tuple[NDArray[np.float64], NDArray[np.float64]]:
    K_x = get_sobel_kernel(x_order=1, y_order=0)
    K_y = get_sobel_kernel(x_order=0, y_order=1)
    I_x = convolve(image, K_x, stride=1)
    I_y = convolve(image, K_y, stride=1)
    # calculate edge strength
    E = np.sqrt(I_x**2 + I_y**2)
    # calculate orientation
    phi = np.arctan2(I_y, I_x)
    return (E, phi)


def non_maximum_suppression(
    magnitude: NDArray[np.float64], orientation: NDArray[np.float64]
) -> NDArray[np.float64]:
    assert magnitude.ndim == orientation.ndim
    assert magnitude.shape == orientation.shape
    C, H, W = magnitude.shape
    # Only for grayscale images
    assert C == 1
    nms = np.zeros(shape=(1, H, W), dtype=np.float64)
    for i in range(1, H - 1):
        for j in range(1, W - 1):
            # Domain of angle is from -180 to 180 degree
            angle = np.rad2deg(orientation[0, i, j])
            # keep angle between [-180,180)
            angle = (
                np.floor(angle / 360) - 180
            )  # Normalize x, x = x_min + (x - x_mean) / (x_max - x_min)
            # Horizontal 0
            if -22.5 < angle <= 22.5 or angle > 157.5 or angle <= -157.5:
                p = magnitude[0, i, j - 1]
                r = magnitude[0, i, j + 1]
            # Diagonal 45
            elif 22.5 < angle <= 67.5 or -157.5 < angle <= -112.5:
                p = magnitude[0, i - 1, j + 1]
                r = magnitude[0, i + 1, j - 1]
            # Vertical
            elif 67.5 < angle <= 112.5 or -112.5 < angle <= -67.5:
                p = magnitude[0, i - 1, j]
                r = magnitude[0, i + 1, j]
```

```
            # Diagonal 135
            else:
                p = magnitude[0, i - 1, j - 1]
                r = magnitude[0, i + 1, j + 1]
            # Non-max suppression
            if magnitude[0, i, j] > p and magnitude[0, i, j] > r:
                nms[0, i, j] = magnitude[0, i, j]
            else:
                nms[0, i, j] = 0

    return nms


def double_thresholding(
    image: NDArray[np.uint8],
    low_threshold_ratio: np.float64 = 0.05,
    high_threshold_ratio: np.float64 = 0.09,
) -> Tuple[NDArray[np.uint8], np.float64, np.float64]:
    high_threshold = np.max(image) * high_threshold_ratio
    low_threshold = high_threshold * low_threshold_ratio
    C, H, W = image.shape
    double_threshold = np.zeros(shape=(C, H, W), dtype=np.uint8)
    strong_ind = np.where(image > high_threshold)
    weak_ind = np.where((image < high_threshold) & (image > low_threshold))
    weak = np.uint8(25)
    strong = np.uint8(255)
    double_threshold[strong_ind] = strong
    double_threshold[weak_ind] = weak
    return (double_threshold, weak, strong)


def edge_tracking_by_hysteresis(
    image: NDArray[np.uint8], weak: np.uint8, strong: np.uint8 = 255
) -> NDArray[np.uint8]:
    C, H, W = image.shape
    edge_track = np.zeros(shape=(C, H, W), dtype=np.uint8)
    for i in range(1, H - 1):
        for j in range(1, W - 1):
            if image[0, i, j] == weak:
                # Check surrounding pixels if any is strong
                try:
                    if (
                        image[0, i - 1, j - 1] == strong
                        or image[0, i - 1, j] == strong
                        or image[0, i - 1, j + 1] == strong
                        or image[0, i, j - 1] == strong
                        or image[0, i, j + 1] == strong
                        or image[0, i + 1, j - 1] == strong
                        or image[0, i + 1, j] == strong
                        or image[0, i + 1, j + 1] == strong
                    ):
                        edge_track[0, i, j] = strong
                    else:
                        edge_track[0, i, j] = 0
                except IndexError as e:
                    pass
            else:
                edge_track[0, i, j] = image[0, i, j]
    return edge_track


def canny(image: NDArray[np.uint8]) -> NDArray[np.uint8]:
    grayscale_image = rgb2grayscale(image)
    img1 = filter_gaussian(grayscale_image, kernel_size=5, sigma=2.5)  # noise reduction
    E, phi = calculate_gradient(img1)
    nms = non_maximum_suppression(E, phi)
    Z, weak, strong = double_thresholding(nms)
    edge_track = edge_tracking_by_hysteresis(Z, weak, strong)
    return edge_track


if __name__ == "__main__":
    image = Image.open("../data/bird.JPEG")
    image_array = np.array(image).transpose((2, 0, 1))
    canny_image = canny(image_array)
    fig = plt.figure(figsize=(10, 7))
    plt.subplot(1, 2, 1)
    plt.imshow(image_array.transpose(1, 2, 0), cmap="gray", vmin=0, vmax=255)
    plt.subplot(1, 2, 2)
    plt.imshow(canny_image.transpose(1, 2, 0), cmap="gray", vmin=0, vmax=255)
    plt.show()
```

2. Output:



3. Code: filters.py

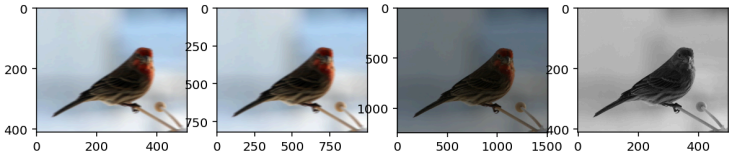
```python
# -*- coding: utf-8 -*-
"""
Created on Fri Jun 19 16:07:25 2026

@author: reina
"""

import numpy as np
from numpy.typing import NDArray


def get_gaussian_kernel(
    kernel_size: int, num_channels: int, sigma: float
) -> NDArray[np.float64]:
    assert kernel_size % 2 == 1  # only odd size filters
    kernel = np.zeros(shape=(kernel_size, kernel_size))
    center_x, center_y = kernel_size // 2, kernel_size // 2
    for i in range(kernel.shape[0]):
        for j in range(kernel.shape[1]):
            kernel[i, j] = 1 / np.exp(
                (i - center_x) ** 2 + (j - center_y) ** 2 / (2 * sigma**2)
            )
    # add a dimension in the first axis to get a kernel of dimension (C, K, K)
    return np.repeat(kernel[np.newaxis, :, :], repeats=num_channels, axis=0) / np.sum(
        kernel
    )


def get_sobel_kernel(x_order: int, y_order: int) -> NDArray[np.int8]:
    if x_order == 1 and y_order == 0:
        kernel = np.array([[1, 2, 1], [0, 0, 0], [-1, -2, -1]])
        return np.repeat(kernel[np.newaxis, :, :], repeats=1, axis=0)
    elif x_order == 0 and y_order == 1:
        kernel = np.array([[-1, 0, 1], [-2, 0, 2], [-1, 0, 1]])
        return np.repeat(kernel[np.newaxis, :, :], repeats=1, axis=0)
    elif x_order == 1 and y_order == 1:
        raise ValueError("Currently, this order combination is not supported.")
    else:
        raise ValueError(
            "Currently, this function does not support orders smaller or greater than 1."
        )


def get_prewitt_kernel(x_order: int, y_order: int) -> NDArray[np.int8]:
    if x_order == 1 and y_order == 0:
        kernel = np.array([[1, 1, 1], [0, 0, 0], [-1, -1, -1]])
        return np.repeat(kernel[np.newaxis, :, :], repeats=1, axis=0)
    elif x_order == 0 and y_order == 1:
        kernel = np.array([[-1, 0, 1], [-1, 0, 1], [-1, 0, 1]])
        return np.repeat(kernel[np.newaxis, :, :], repeats=1, axis=0)
    elif x_order == 1 and y_order == 1:
        raise ValueError("Currently, this order combination is not supported.")
    else:
        raise ValueError(
            "Currently, this function does not support orders smaller or greater than 1."
        )
```

4. Code: convolution.py

```python
# -*- coding: utf-8 -*-
"""
Created on Fri Jun 19 16:10:01 2026

@author: reina
"""

from PIL import Image
import numpy as np
import matplotlib.pyplot as plt
from numpy.typing import NDArray
from filters import get_gaussian_kernel
from utils import *

# Images are C,H,W format
```

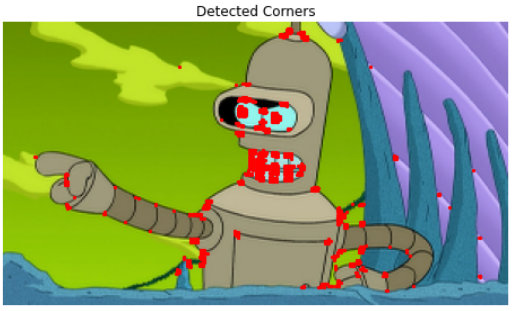
```
"""
References:
1. https://stackoverflow.com/questions/66287552/how-to-implement-fractionally-strided-convolution-layers-in-
pytorch
2. https://stackoverflow.com/questions/69782823/understanding-the-pytorch-implementation-of-conv2dtranspose
3. https://stats.stackexchange.com/questions/297678/how-to-calculate-optimal-zero-padding-for-convolutional-
neural-networks

"""


def convolve(
    image: NDArray[np.uint8], kernel: NDArray[np.float64], stride: int
) -> NDArray[np.float64]:
    # flip kernel
    kernel = np.flipud(np.fliplr(kernel))
    # Handle grayscale image
    if image.ndim == 2:
        image = image[np.newaxis, ...]
    _, kernel_H, kernel_W = kernel.shape
    C, H, W = image.shape
    P_x = np.int_(
        np.ceil(((stride - 1) * H - stride + kernel_H) / 2)
    )  # required padding on input to get the same padding on output
    P_y = np.int_(np.ceil(((stride - 1) * W - stride + kernel_W) / 2))
    # apply paddding on input image
    # image = np.pad(
    #     image, pad_width=((0, 0), (P_x, P_x), (P_y, P_y)), mode="constant"
    # )  # pad top and bottom side by P_x each,
    image = np.pad(
        image, pad_width=((0, 0), (P_x, P_x), (P_y, P_y)), mode="reflect"
    )  # pad top and bottom side by P_x each,
    assert kernel_H <= H and kernel_W <= W  # pad left and right side by P_y each
    output_H = (H - kernel_H + 2 * P_x) // stride + 1
    output_W = (W - kernel_W + 2 * P_y) // stride + 1
    convolved = np.zeros(shape=(C, output_H, output_W), dtype=np.float64)
    print("convolved.shape: ", convolved.shape)
    for i in range(output_H):
        for j in range(output_W):
            h_start = i * stride
            h_end = h_start + kernel_H
            w_start = j * stride
            w_end = w_start + kernel_W
            patch = image[:, h_start:h_end, w_start:w_end]
            product = np.multiply(patch, kernel)  # np.multiply performs element-wise multiplication
            convolved[:, i, j] = product.sum(axis=(1, 2))
    # normalization can be called separately
    # return normalize_image(convolved)
    return convolved


def convolve_transpose(
    image: NDArray[np.uint8], kernel: NDArray[np.float64], stride: int = 1
) -> NDArray[np.float64]:
    # Handle grayscale image
    if image.ndim == 2:
        image = image[np.newaxis, ...]
    _, kernel_height, kernel_width = kernel.shape
    C, H, W = image.shape
    output_H = (H - 1) * stride + kernel_height
    output_W = (W - 1) * stride + kernel_width
    convolved_transposed = np.zeros(shape=(C, output_H, output_W), dtype=np.float64)
    # # calculate the output
    # for i in range(image_height):
    #     for j in range(image_width):
    #         for k_row in range(kernel_height):
    #             for k_col in range(kernel_width):
    #                 i_out = i*stride+k_row
    #                 j_out = j*stride+k_col
    #                 convolved_transposed[:,i_out, j_out] += kernel[:,k_row,k_col]*image[:,i,j]
    # print("output_height,output_width",output_height,output_width)
    for i in range(H):
        for j in range(W):
            h_start = i * stride
            h_end = h_start + kernel_height
            w_start = j * stride
            w_end = w_start + kernel_width
            convolved_transposed[:, h_start:h_end, w_start:w_end] += (
                image[:, i, j].reshape((C, 1, 1)) * kernel
            )  # scalar*kernel, element-wise multiply

    # desired_height = image_height*stride
    # desired_width = image_width*stride
    # normalization can be called separately
    return convolved_transposed


if __name__ == "__main__":
    image = Image.open("n01532829_1261.JPEG")
```

```
    image_array = np.array(image).transpose((2, 0, 1))
    print("image array _shape: ", image_array.shape)

    # image_array = np.array([[2,4],
    #                         [0,1]])
    # image_array = np.array([[[2,4],
    #                         [0,1]]])
    # image_array = np.random.randint(5,size=(1,10,10))
    print("image_array.shape ", image_array.shape)
    # image_array = np.repeat(image_array,repeats=3,axis=2)
    # print(image_array.shape)
    # print(array)
    # plt.imshow(array)
    # plt.show()
    # stride = 1
    kernel = get_gaussian_kernel(kernel_size=11, num_channels=3, sigma=500)
    # kernel = np.array([[[3,1],
    #                     [1,5]]])
    # kernel = np.random.randint(5,size=(1,5,5))
    # kernel = np.repeat(kernel,repeats=3,axis=0)
    print("kernel shape : ", kernel.shape)

    convolved_image = normalize_image(convolve(image_array, kernel, stride=1), 0, 255)
    # convolved_image = normalize_image(convolve(image_array,kernel=kernel,stride=2))  # tiling effect for
stride>1 (because of reflect padding)
    # convolved_image = convolve(image_array,kernel,stride)
    print(convolved_image.shape)
    # print("convolved_image.shape ", convolved_image.shape)
    upsampled_image = bilinear_interpolation(convolved_image, scale=2)
    # # upsampled_image = upsample_nearest_neighbor(image_array,stride=2)
    # # convolved_transposed_image = convolve_transpose(convolved_image,kernel,stride=2)
    convolved_transposed_image = normalize_image(
        convolve_transpose(image_array, kernel, stride=3), 0, 255
    )  # Transposed convolution is also known as fractionally strided convolution, the actual stride is 1/3
    # # convolved_transposed_image = normalize_image(convolve_transpose(image_array,kernel,stride=3))
    # # convolved_transposed_image = normalize_image(convolve_transpose(image_array,kernel,stride=5))
    grayscale_image = normalize_image(rgb2grayscale(image_array), 0, 255)
    # # grayscale_image = normalize_image(rgb2grayscale(image_array))
    # # convolved_transposed_image = convolve_transpose(image_array,kernel,stride=3)
    # # print("convolved transposed image :", convolved_transposed_image)
    # print("image_array shape: ", image_array.shape)
    # print("convolved transposed shape: ", convolved_transposed_image.shape)
    # print("image_array max: ", np.max(image_array, axis=(1, 2), keepdims=True))
    # print("image_array median: ", np.median(image_array, axis=(1, 2)))
    # print(
    #     "convolved_transposed_image max: ",
    #     np.max(convolved_transposed_image, axis=(1, 2)),
    # )
    # print(
    #     "convolved_transposed_image median: ",
    #     np.median(convolved_transposed_image, axis=(1, 2)),
    # )
    # # print("grayscale_image max: ",np.max(grayscale_image,keepdims=True))

    # # plt.show()
    fig = plt.figure(figsize=(10, 7))
    plt.subplot(1, 4, 1)
    plt.imshow(convolved_image.transpose(1, 2, 0))
    plt.subplot(1, 4, 2)
    plt.imshow(upsampled_image.transpose(1, 2, 0))
    # # plt.imshow(image_array.transpose(1,2,0),origin="lower")
    # # plt.imshow(image_array.transpose(1,2,0))
    plt.subplot(1, 4, 3)
    plt.imshow(convolved_transposed_image.transpose(1, 2, 0))
    plt.subplot(1, 4, 4)
    plt.imshow(grayscale_image.transpose(1,2,0), cmap='gray', vmin=0, vmax=255)
    # # axarr[0].imshow(convolved_image)
    # # axarr[1].imshow(upsampled_image)
    plt.show()
```

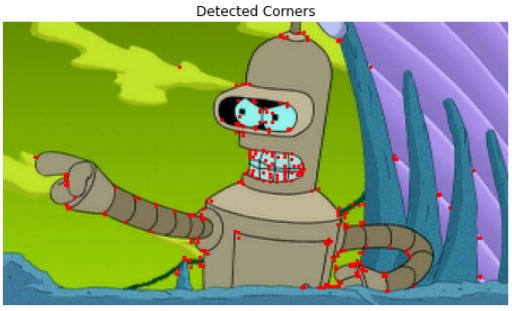
5. Output:
6. 

2. Harris corner detection
    1. Gradient computation
        1. Compute the gradients I_x and I_y using the Sobel operators:
        2. $I_x = \begin{pmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{pmatrix}$ and $\begin{pmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{pmatrix}$

    3. Convolve the grayscale image with I_x and I_y to obtain the gradient images I_x and I_y.
2. Structure Tensor calculation
    1. Compute the elements of the structure tensor:
    2. I_x^2, I_y^2, and $I_x \cdot I_y$
    3. These computations involve element-wise multiplication of I_z and I_y with themselves.
3. Harris response calculation
    1. For each pixel (i, j), compute the elements of the structure tensor within a local window of size $N \times N$. Compute the sums of squares of I_x and I_y within the window to obtain S_{xy}. Compute the determinant and trace of structure tensor:
    2. $\det(M) = S_{xx} \times S_{yy} - S_{xy}^2$, $\mathrm{Tr}(M) = S_{xx} + S_{yy}$
    3. Compute the Harris response (R) using the formula:
    4. $R = \det(M) - k \times (\mathrm{Tr}(M))^2$
    5. where k is an empirically chosen constant.
4. Thresholding and Non-maximum Suppression
    1. Apply a threshold to R to identify potential corner candidates. Perform non-maximum suppression to retain only local maxima as corner points.
5. Visualization
    1. Plot the original image with detected corner points marked.
6. Before non-max suppression



7. After non-max suppression



8. Code: harris.py

```
from PIL import Image
import numpy as np
import matplotlib.pyplot as plt
from filters import *
from utils import *

from typing import Tuple
from convolution import convolve
from canny import filter_gaussian


"""
Harris Corner Detection

1. Convert RGB image to grayscale.
2. Noise reduction.
3. Compute first derivatives, I_x and I_y.
4. Compute second derivatives, I_x^2, I_y^2 and I_x*I_y.
5. Compute structure tensor.
6. Compute cornerness score.
7. Non-maximum suppression.


Translation Invariance: It is translation invariant, as the eigenvalues of translated corner remain the same.
Rotation Invariance: It is rotation invariant. Direction of eigenvectors change but the eigenvalues remain the
same.
```

```
Scale Invariance: Not invariant to scaling as the points become edges when scaled up.

References:
8. C.Harris and M.Stephens. “A Combined Corner and Edge Detector.”1988
9. https://www.cs.cmu.edu/~16385/s17/Slides/6.2_Harris_Corner_Detector.pdf
10. https://www.cs.cornell.edu/courses/cs5670/2021sp/lectures/lec04_harris_for_web.pdf
11. https://stackoverflow.com/questions/51740695/non-maximum-suppression-in-corner-detection

"""



def get_first_derivatives(
    image: NDArray[np.uint8],
) -> Tuple[NDArray[np.float64], NDArray[np.float64]]:
    assert image.ndim == 3
    num_channels, _, _ = image.shape
    assert num_channels == 1
    K_x = get_prewitt_kernel(x_order=1, y_order=0)
    K_y = get_prewitt_kernel(x_order=0, y_order=1)
    I_x = convolve(image, K_x, stride=1)
    I_y = convolve(image, K_y, stride=1)
    return (I_x, I_y)


def get_second_derivatives(
    I_x: NDArray[np.float64], I_y: NDArray[np.float64]
) -> Tuple[NDArray[np.float64], NDArray[np.float64], NDArray[np.float64]]:
    assert I_x.shape == I_y.shape
    I_x2 = np.multiply(I_x, I_x)  # Hadamard product
    I_y2 = np.multiply(I_y, I_y)
    I_xI_y = np.multiply(I_x, I_y)
    return (I_x2, I_y2, I_xI_y)


def compute_sums_of_product_of_derivatives(
    I_x2: NDArray[np.float64], I_y2: NDArray[np.float64], I_xI_y: NDArray[np.float64]
) -> Tuple[NDArray[np.float64], NDArray[np.float64], NDArray[np.float64]]:
    gaussian_kernel = get_gaussian_kernel(kernel_size=5, num_channels=1, sigma=1.4)
    S_xx = convolve(I_x2, gaussian_kernel, stride=1)
    S_yy = convolve(I_y2, gaussian_kernel, stride=1)
    S_xy = convolve(I_xI_y, gaussian_kernel, stride=1)
    return (S_xx, S_yy, S_xy)


def compute_R_matrix(S_xx, S_yy, S_xy):
    assert S_xx.shape == S_yy.shape == S_xy.shape
    C, H, W = S_xx.shape
    assert C == 1
    R = np.zeros(shape=S_xx.shape, dtype=np.float64)
    for i in range(H):
        for j in range(W):
            try:
                # compute M
                M = np.zeros(shape=(2, 2), dtype=np.float64)
                M[0, 0] = S_xx[0, i, j]
                M[1, 1] = S_yy[0, i, j]
                M[0, 1] = S_xy[0, i, j]
                M[1, 0] = S_xy[0, i, j]
                k = 0.04  # k is empirically 0.04-0.06
                # compute Harris corner score R
                R[0, i, j] = np.linalg.det(M) - k * np.trace(M) ** 2
                # print(R[0,i,j])
            except IndexError as e:
                pass
    return R


def get_corner_response(R: NDArray[np.float64]) -> NDArray[np.float64]:
    # large positive responses correspond to corners
    C, H, W = R.shape
    assert C == 1
    max_val = np.max(R, axis=(1, 2))
    threshold = 0.001 * max_val
    out = np.zeros(shape=R.shape, dtype=np.float64)
    for i in range(H):
        for j in range(W):
            if R[0, i, j] > threshold:  # positive responses correspond to corners
                out[0, i, j] = 255  # set high
    return out


def get_edge_response(R: NDArray[np.float64]) -> NDArray[np.float64]:
    # negative responses correspond to edges
    C, H, W = R.shape
    assert C == 1
    min_val = np.min(R, axis=(1, 2))
    threshold = 0.001 * min_val
    out = np.zeros(shape=R.shape, dtype=np.float64)
```

```
    for i in range(H):
        for j in range(W):
            if R[0, i, j] < threshold:  # negative responses correspond to edges
                out[0, i, j] = 255  # set high
    return out


def draw(
    image: NDArray[np.uint8],
    nms: NDArray[np.float64],
    color: Tuple[np.uint8, np.uint8, np.uint8],
):
    assert image.ndim == 3
    assert image.shape[0] == 3  # RGB
    C, H, W = nms.shape
    # cropped_image=center_crop(image,nms.shape[1],nms.shape[2]).copy()
    img_cpy = image.copy()
    for i in range(H):
        for j in range(W):
            if nms[0, i, j] > 0:
                # img_cpy[:,i,j]=color
                img_cpy = draw_circle(img_cpy, (i, j), 2, color, 1)
    return img_cpy


def non_maximum_suppression(R: NDArray[np.float64]):
    assert R.ndim == 3
    C, H, W = R.shape
    assert C == 1
    cell_size = 10  # partition response into tiles of size cell_size x cell_size,
    # also corresponds to the minimum separation space between features.
    num_cell_h = (H + cell_size - 1) // cell_size
    num_cell_w = (W + cell_size - 1) // cell_size
    out = np.zeros(shape=R.shape, dtype=np.float64)
    for i in range(num_cell_h):
        for j in range(num_cell_w):
            # take top n largest responses
            n = 5
            h_start = i * cell_size
            h_end = h_start + cell_size
            w_start = j * cell_size
            w_end = w_start + cell_size
            tile = R[0, h_start:h_end, w_start:w_end]
            if np.max(tile) == 0:
                continue
            x, y = np.unravel_index(
                np.argsort(tile.ravel(), axis=None)[::-1][:n], tile.shape
            )
            for x_, y_ in zip(x, y):
                out[0, h_start + x_, w_start + y_] = R[0, h_start + x_, w_start + y_]
    return out


# def
draw_edge_response(image:NDArray[np.uint8],nms:NDArray[np.float64],color:Tuple[np.uint8,np.uint8,np.uint8]):
#     assert image.ndim==3
#     assert image.shape[0]==3  # RGB
#     C,H,W=nms.shape
#     cropped_image=center_crop(image,nms.shape[1],nms.shape[2]).copy()
#     for i in range(H):
#         for j in range(W):
#             if nms[0,i,j]<0:
#                 cropped_image[:,i,j]=color  # Trurqoise
#     return cropped_image

if __name__ == "__main__":
    image = Image.open("../data/bird.JPEG")
    # image = Image.open("../data/n01855672_232.JPEG")
    image_array = np.array(image).transpose((2, 0, 1))

    img1 = filter_gaussian(image_array, kernel_size=5, sigma=1.4)
    grayscale_image = rgb2grayscale(img1)
    I_x, I_y = get_first_derivatives(grayscale_image)
    print("I_x.shape,I_y.shape", I_x.shape, I_y.shape)
    I_x2, I_y2, I_xI_y = get_second_derivatives(I_x, I_y)
    print("I_x2.shape,I_y2.shape,I_xI_y.shape", I_x2.shape, I_y2.shape, I_xI_y.shape)
    S_xx, S_yy, S_xy = compute_sums_of_product_of_derivatives(I_x2, I_y2, I_xI_y)
    print("S_xx.shape,S_yy.shape,S_xy.shape", S_xx.shape, S_yy.shape, S_xy.shape)
    R = compute_R_matrix(S_xx, S_yy, S_xy)
    print("R.shape", R.shape)
    corner_response = get_corner_response(R)
    edge_response = get_edge_response(R)
    C = non_maximum_suppression(corner_response)
    E = non_maximum_suppression(edge_response)
    print("C.shape", C.shape)
    print("E.shape", E.shape)
    print("np.max(C)", np.max(C))
    print("np.max(E)", np.max(E))
    print("corner_response.shape", corner_response.shape)
    print("edge_response.shape", edge_response.shape)
```

```
    out_corner = draw(image_array, C, (255, 0, 0))
    out_edge = draw(image_array, E, (48, 213, 200))

    fig = plt.figure(figsize=(10, 7))
    # plt.subplot(1,4,1)
    # plt.imshow(image_array.transpose(1,2,0))
    plt.subplot(1, 3, 1)
    plt.imshow(edge_response.transpose(1, 2, 0), cmap="gray", vmin=0, vmax=255)
    plt.subplot(1, 3, 2)
    plt.imshow(out_corner.transpose(1, 2, 0))
    plt.subplot(1, 3, 3)
    plt.imshow(out_edge.transpose(1, 2, 0))
    # plt.imshow(gx.transpose(1,2,0),cmap="gray",vmin=0,vmax=255)
    plt.show()
```

9. Code:

```
# -*- coding: utf-8 -*-
"""
Created on Tue Feb 17 22:10:18 2026

@author: reina
"""

from scipy.ndimage import convolve
from skimage import color
from skimage import io
import numpy as np
import cv2
import matplotlib.pyplot as plt

def harris_corner_detection(image, threshold=0.01, window_size=3, k=0.04):
    # Convert the image to grayscale
    gray = color.rgb2gray(image)

    # Compute gradients using Sobel operators
    Ix = convolve(gray, np.array([[-1, 0, 1], [-2, 0, 2], [-1, 0, 1]]))
    Iy = convolve(gray, np.array([[-1, -2, -1], [0, 0, 0], [1, 2, 1]]))

    # Compute elements of the structure tensor
    Ix2 = Ix * Ix
    Iy2 = Iy * Iy
    Ixy = Ix * Iy

    # Compute Harris response
    height, width = gray.shape
    R = np.zeros((height, width))
    offset = window_size // 2
    for i in range(offset, height - offset):
        for j in range(offset, width - offset):
            Sxx = np.sum(Ix2[i - offset:i + offset + 1, j - offset:j + offset + 1])
            Syy = np.sum(Iy2[i - offset:i + offset + 1, j - offset:j + offset + 1])
            Sxy = np.sum(Ixy[i - offset:i + offset + 1, j - offset:j + offset + 1])
            det = Sxx * Syy - Sxy**2
            trace = Sxx + Syy
            R[i, j] = det - k * trace**2

    # Thresholding
    corners = np.zeros_like(R)
    corners[R > threshold * np.max(R)] = 1
    # Perform non-maximum suppression
    corners = non_maximum_suppression(corners)

    return corners

def non_maximum_suppression(corners):
    # Define 8-neighbor offsets
    offsets = [(i, j) for i in [-1, 0, 1] for j in [-1, 0, 1] if (i != 0 or j != 0)]

    # Create a copy of the corners matrix
    suppressed_corners = corners.copy()

    # Iterate over each pixel in the corners matrix
    height, width = corners.shape
    for i in range(height):
        for j in range(width):
            # Check if the current pixel is a corner
            if corners[i, j] > 0:
                # Calculate the sum of the values of the neighboring pixels
                neighbor_sum = 0
                for offset_i, offset_j in offsets:
                    ni, nj = i + offset_i, j + offset_j
                    if ni >= 0 and ni < height and nj >= 0 and nj < width:
                        neighbor_sum += corners[ni, nj]

                # If the sum of neighboring pixels is less than 4, keep the current pixel as a corner
                if neighbor_sum < 4:
                    suppressed_corners[i, j] = 1
                else:
                    suppressed_corners[i, j] = 0
```

```
    return suppressed_corners

# Load the example image
img = cv2.imread('bender.png')
img = cv2.resize(img, (300, 169), interpolation=cv2.INTER_CUBIC)
#plt.imshow(img, cmap='gray')

# Perform Harris corner detection
corners = harris_corner_detection(img)

# Visualize the detected corners
plt.figure(figsize=(8, 6))
plt.imshow(img)
plt.scatter(np.nonzero(corners)[1], np.nonzero(corners)[0], color='r', s=5)
plt.title('Detected Corners')
plt.axis('off')
plt.savefig('harris.png')
plt.show()
```

3. SIFT
    1. People say 'SIFT' but there are two parts to SIFT [1].
        1. an interest point detector.
        2. a region descriptor.
            1. They are independent, many people use the region descriptor without looking for the points.
    2. SIFT summary [2]:
        1. Extract a 16x16 patch around detected keypoint.
        2. Compute the gradients and apply a Gaussian weighting function.
        3. Divide the window into a 4x4 grid of cells.
        4. Compute gradient direction histograms over 8 directions in each cell.
        5. Concatenate the histograms over 8 directions in each cell.
        6. Concatenate the histograms to obtain 128 dimensional feature vector.
        7. Normalize to unit length.

	Rotation Invariant	Scale Invariant	Affine Invariant	Repeatability	Localization Accuracy	Robustness	Efficiency
Haris	x			+++	+++	++	++
Shi-Tomasi	x			+++	+++	++	++
FAST	x	x		++	++	++	++++
SIFT	x	x	x	+++	++	+++	+
SURF	x	x	x	+++	++	++	++

Image Courtesy of Davide Scaramuzza et. al 2012.

        8. Local features: main components [3]
            1. Detection: Identify the interest points.
            2. Description: Extract vector feature descriptor surrounding each interest point.
            3. Matching: Determine correspondence between descriptors in two views.
4. ORB
    1. FAST (detector), BRIEF (descriptor)
    2. ORB (Oriented FAST and Rotated BRIEF) combines FAST and BRIEF, adds some orientation to the FAST and adds some rotation to the BRIEF, so this will make it more robust.
5. Optical flow
    1. Motivation: we want to match features between two frames, however we don't want to perform feature matching from one view to another view. Can we just first detect some of the key points or interest points corners, edges, etc, and then use some algorithm to know the correspondences in the next image without searching for the correspondences. Because this searching for correspondences consumes a lot of time. A more efficient way is to use algorithm to compute the locations of these correspondences in the next frame, thus we can form correspondences in this approach. This algorithm is called Lucas-Kanade algorithm (LK algorithm).
    2. Estimate the motion of each pixel between two consecutive image frames, based on assumptions:

    1. brightness consistency
    2. small motion
3. Can we use accelerometer instead? yes, however acceleration needs to be integrated twice to obtain position.
4. with optical flow we have one less integration error.
5. we want to estimate pixel motion from image I(x, y, t) to image I(x, y, t+1).
6. Optical flow: *Apparent* motion of objects.
7. examples where:
    1. There is no optical flow, but there is relative motion (between camera and environment): Could happen in textureless environment.
    2. There is no relative motion, but there is optical flow: changing light source or intensity, the sun or some other source of light, the camera is fixed relative to the environment but you can see apparent change of the environment.
8. Apparent motion could be due to different factors:
    1. Motion of object in camera's view.
    2. Motion of camera itself.
    3. Lighting changes.
9. Birds rely on trick of the eye: optical flow. Optical flow cues in birds. Birds tend to keep similar optical flow on the left and right eyes . Birds use optical flow to fine-tune their navigation when manuevering through narrow spaces at high speeds.

References:

1. CSE576 Udub Linda Shapiro
2. Trym Vegard Haavardsholm, UNIK4690
3. CSIE 5429 3D-DLCV, 3D Computer Vision with Deep Learning Applications, National Taiwan University, Spring 2021.
4. Lecture 18, Princeton, Introduction to Robotics, Optical flow.